

IEEE SYSTEMS, MAN, AND CYBERNETICS SOCIETY
STUDENT BRANCH CHAPTER
KALYANI GOVERNMENT ENGINEERING COLLEGE
(KGEC)

SUMMER RESEARCH INTERNSHIP PROGRAMME
2026

FINAL RESEARCH REPORT — SUBMISSION COVER PAGE

PROJECT / RESEARCH TITLE

Thunderstorm Prediction Using Virtual Weather Sensor Nodes and Machine Learning

PROGRAMME DETAILS

Domain / Track:	<i>Artificial Intelligence / Machine Learning – IoT</i>	Duration:	<i>15th May, 2026 – 14th August, 2026</i>
Department:	<i>Computer Science & Engineering</i>	Batch / Year:	<i>2023 – 2027 (Final Year)</i>

SUBMITTED BY

Student 1 *Pranjal Mohata*

UNDER THE MENTORSHIP OF

Mentor Name *Anuradha Banerjee*
Affiliation *Department of CA, Kalyani Government Engineering College*

DECLARATION

We hereby declare that this report is a record of genuine work carried out by us under the guidance of the above-mentioned mentor, as part of the IEEE SMC Student Branch Chapter, KGEC — Research Internship Programme 2026, and has not been submitted elsewhere for any other purpose.

Anuradha Banerjee

Mentor's Signature

Programme Coordinator

IEEE SMC Student Branch Chapter, KGEC | Research Internship Programme 2026 | Date of Submission: [18 / 08 / 2026]



Final Report of Internship Programme

THUNDERSTORM PREDICTION USING VIRTUAL WEATHER SENSOR NODES AND MACHINE LEARNING

Abstract

Thunderstorms strike fast and can throw local weather into disarray within minutes, raising real safety concerns and putting infrastructure at risk. This project builds a software-defined system to watch for that risk without relying on physical hardware — instead, a set of virtual sensor nodes pull live weather data from an online API for several fixed locations. That data gets logged to a CSV file, a handful of engineered features are computed from it (pressure change and humidity change turned out to matter most), and a RandomForest model is trained to flag thunderstorm versus non-thunderstorm conditions. The model is then wired into a small Flask dashboard so each node's current weather and predicted risk can be viewed at a glance. One addition made after an earlier round of feedback: nodes that are flagged as at-risk now get pointed to the nearest node that is currently safe, which pushes the dashboard from something purely informational toward something a user could actually act on.

Keywords: *Internet of Things, Virtual Sensor Nodes, Thunderstorm Prediction, Machine Learning, RandomForest, OpenWeatherMap API, Flask Dashboard, Leaflet Map, Safe-Node Recommendation.*

1. Introduction

Thunderstorms don't build up slowly. High wind, sudden cloud formation, sharp pressure swings, and heavy rainfall can all appear within a short window, which is exactly what makes them hard to plan around. If a system can catch the early signs of a thunderstorm-prone condition, it becomes possible to raise alerts, keep an eye on local risk, and give people at least a rough sense of what's coming — something that matters a lot for smart-city setups and IoT-based monitoring in general.

Rather than deploying physical weather stations, this project takes a distributed-sensing approach in software: a set of virtual sensor nodes, each tied to a fixed location, pulls live readings from an online weather Application Programming Interface (API). The resulting pipeline covers data acquisition, labelling, feature extraction, model training, saving the trained model, and finally a dashboard the user can actually look at. None of this needs custom hardware to get started.

That trade-off is deliberate. A hardware prototype takes longer to build and is harder to iterate on, whereas this API-driven version can be stood up quickly and still leaves room to swap in ESP32-based or other physical sensors later if the project moves in that direction. One thing that shaped the current version directly: earlier feedback pointed out that the dashboard, while functional, was purely descriptive — it told you a node was at risk and stopped there. In response, the dashboard now also looks for the nearest node currently predicted as safe and suggests it. The system, called TempestGuard, is meant to move from reporting risk to actually helping someone decide what to do about it.

2. Objective

Broadly, this project sets out to build a software-defined system for thunderstorm-risk prediction. More concretely, the goals were to:

- Collect live weather observations from multiple virtual sensor nodes using a public weather API;
- Engineer temporal features that capture short-term atmospheric variation, such as pressure change and humidity change;

Final Report of Internship Programme

- Train and evaluate a machine learning classifier to distinguish thunderstorm from non-thunderstorm conditions;
- Deploy the trained model through a live, node-wise monitoring dashboard; and
- Extend the dashboard with a nearest safe-node recommendation module so that the system provides actionable, decision-support guidance rather than description alone.

3. Literature / Background

Most existing weather-monitoring setups depend on dedicated physical sensor networks or on meteorological-department infrastructure — both of which get expensive fast if you're only trying to cover a campus or a small local area. Public weather-data services have changed that calculus somewhat. The OpenWeatherMap Current Weather Data API [1], for instance, makes it possible to approximate a distributed sensor network entirely in software, just by repeatedly querying fixed coordinates. For a student project, that's a reasonable trade: no hardware dependency, and the data is still real and time-varying rather than simulated.

On the modelling side, ensemble tree methods like RandomForest [2] tend to be a practical default for this kind of tabular weather data — they cope reasonably well with nonlinear relationships and mixed-scale inputs, and they don't demand much preprocessing before training. For getting a model in front of a user, lightweight frameworks such as Flask [3] paired with a mapping library like Leaflet [4] are a fairly standard combination for exposing a model through a dashboard. What this project adds on top of that familiar stack is the nearest safe-node suggestion — most description-only dashboards stop at showing risk and don't take the next step of pointing somewhere safer.

4. Methodology

The system design consists of three layers: a sensor layer, a processing layer, and an output layer, as shown in Fig. 1.

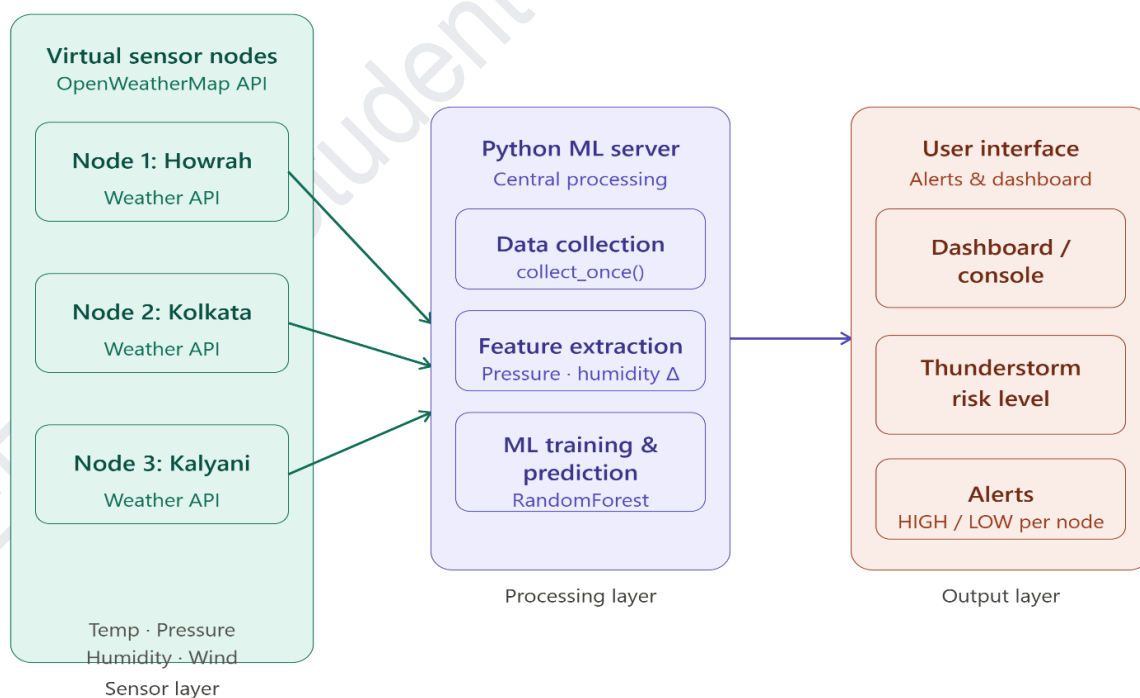


Fig. 1. Overall system architecture showing the sensor, processing, and output layers.

In the sensor layer, multiple virtual nodes fetch live weather observations from the OpenWeatherMap API using latitude and longitude coordinates. In the processing layer, the collected data is appended to a CSV file,

Final Report of Internship Programme

thunderstorm labels are assigned, derived features are computed, and a RandomForest classifier is trained. In the output layer, a Flask application loads the latest records, applies the trained model, and displays node-wise thunderstorm status through a dashboard interface. The collector script defines five nodes — Howrah AC, Howrah Maidan, Burrabazar, Newtown, and Barasat. For each node, the script captures temperature, pressure, humidity, wind speed, cloudiness, rainfall in the previous hour, general weather condition, and weather ID, and stores these records in `thunderstorm_raw_data.csv` for model development.

4.1 System Overview

The working flow begins with repeated weather collection from the configured virtual sensor nodes. Each API response is parsed and transformed into a structured row containing meteorological parameters and a generated binary label, so that the CSV file becomes the historical dataset for the model. During preprocessing, the training script sorts the dataset by node and time, computes the previous pressure and humidity values per node, and creates two engineered features, `pressure_change` and `humidity_change`. Missing change values for the first reading of each node are filled with zero.

The labelled rows are then split into input features and output labels for model training. The model is trained using `RandomForestClassifier` [2] with 100 estimators and a fixed random state, using a 70/30 train-test split. Predictions are evaluated through accuracy and a classification report, and the trained model is stored as `storm_model.pkl`. The dashboard subsequently loads this model and applies it to the latest readings from each node to display "Thunderstorm Risk" or "No Thunderstorm" as the predicted status.

4.2 Data Collection and Automatic Label Generation

The data collection module uses the OpenWeatherMap current-weather API [1]. For every configured node, the script sends a request using latitude and longitude and extracts the main weather statistics from the JSON response, namely `temp`, `pressure`, `humidity`, `wind_speed`, `cloudiness`, `rain_1h`, `weather_main`, and `weather_id`. The class label is generated automatically from this metadata: if `weather_main` equals "Thunderstorm", or if `weather_id` lies between 200 and 299, the record is labelled as 1; otherwise it is labelled as 0. This rule-based labelling provides a practical method for building a supervised dataset from live API records without manual annotation.

4.3 Feature Engineering

The main feature engineering step is based on the temporal change in atmospheric properties. For each node, the previous pressure and humidity values are computed using grouped shifting by node and time, giving two change-based features: `pressure_change = pressure - pressure_prev` and `humidity_change = humidity - humidity_prev`. The final feature set used by the classifier contains eight features in total: temperature, pressure, humidity, wind speed, cloudiness, rainfall in the previous hour, pressure change, and humidity change. These features were selected because they represent both the immediate weather conditions and the short-term atmospheric variation associated with storm onset.

4.4 Machine Learning Model

The classification module uses a RandomForest classifier [2], which is suitable for this task because it can learn nonlinear relationships among weather variables, handles mixed-scale numeric inputs well, and is comparatively easy to train and deploy for an academic prototype. The classifier is trained with 100 trees and evaluated on the held-out test set, as detailed in Fig. 2.

Final Report of Internship Programme

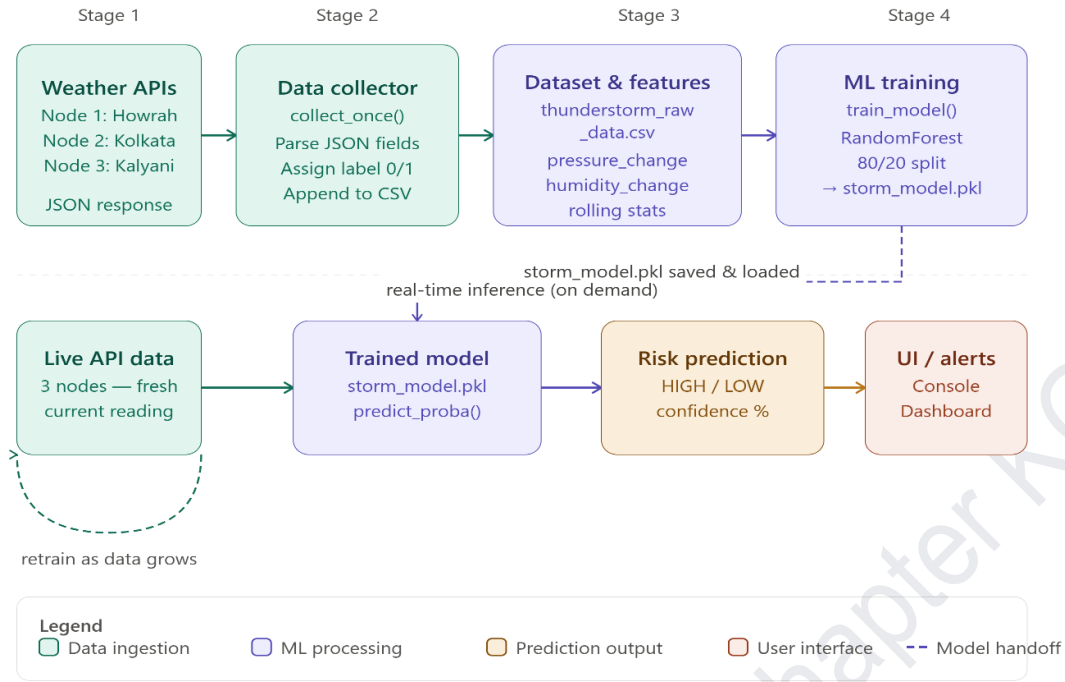


Fig. 2. Methodology and working-principle flow, from data collection to dashboard prediction.

4.5 Dashboard Prediction and Safe-Node Recommendation

The Flask dashboard reads the latest available record per node from the CSV file and prepares the same feature set required by the model for inference. The predicted status is displayed node-wise alongside the current weather condition and other metrics in a dark-themed web interface. The dashboard uses a layout consisting of a top status bar, a central Leaflet map [4], and a right-hand column of node cards; each card shows temperature, pressure, humidity, wind speed, cloudiness, and rainfall for one node, together with a status badge of "No Thunderstorm" or "Thunderstorm Risk". The map displays circular markers for each node at its geographic coordinates.

To implement the nearest safe-place recommendation, the system assigns fixed latitude and longitude coordinates to each node. After obtaining predictions from the model, the dashboard divides the nodes into safe and risky sets. For every risky node, it iterates over all safe nodes and calculates the great-circle distance using the haversine formula; the safe node with the smallest distance is selected as the nearest safest place and shown directly on the node card — for example, "Thunderstorm Risk Detected. Nearest safest place: Newtown (3.42 km away)." If no node is currently predicted as safe, the card instead displays a warning that no mapped safe zone is available and advises the user to move indoors immediately.

5. Results and Discussion

5.1 Dataset and Model Results

The dataset used during training contains a few hundred rows of weather observations collected from the configured nodes, as summarised in Fig. 3. The label distribution is imbalanced, with substantially more non-thunderstorm samples than thunderstorm samples. The evaluation on the held-out test set gives high accuracy, with strong precision, recall, and F1-score for both classes in that split, indicating that the trained model separates the available test examples successfully.

Final Report of Internship Programme

```

PS D:\Thunderstorm> python -u "d:\Thunderstorm\sensor_network_thunderstorm_ml.py"
Dataset size: 260 rows
Label counts: {0: 252, 1: 8}

Model performance:
Accuracy: 1.0

Classification report:

              precision    recall  f1-score   support

     0       1.00        1.00        1.00        75
     1       1.00        1.00        1.00         3

 accuracy          1.00
 macro avg          1.00
 weighted avg       1.00

Model saved to storm model.pkl

```

Fig. 3. Dataset summary and classification performance on the held-out test set.

However, because the thunderstorm class has relatively few examples in the full dataset, this performance should be interpreted with caution. The result is promising for a classroom prototype, but it does not yet guarantee robust real-world generalisation under previously unseen weather conditions.

5.2 User Interface and Output

The user interface is implemented through a Flask web application, shown in Fig. 4. The dashboard loads the most recent weather record for each node, runs prediction using the saved model, and constructs a row-wise display containing node name, time, temperature, pressure, humidity, cloudiness, rainfall, weather condition, and predicted risk.

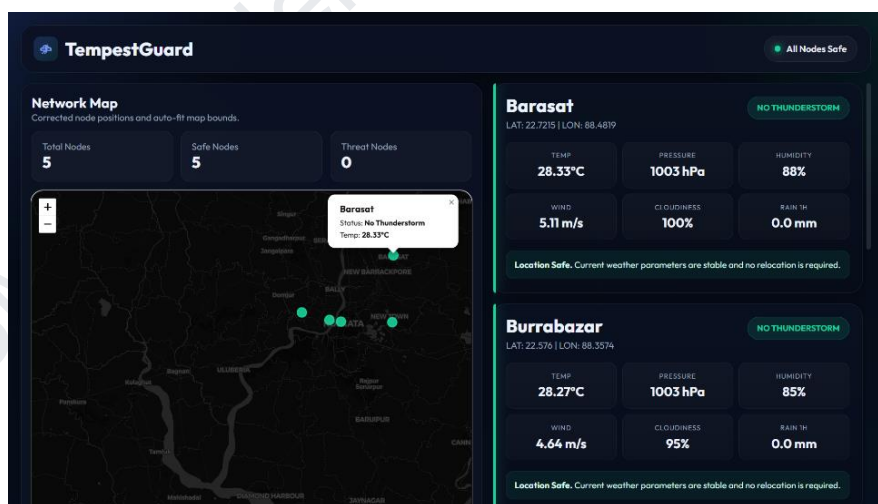


Fig. 4. Dashboard interface showing node-wise weather data and predicted thunderstorm risk.

This interface serves as the output layer of the system and functions as a basic monitoring console. It demonstrates how the machine learning model can be integrated with a live application to transform raw meteorological data into interpretable risk labels for end users.

5.3 Novelty of the Work

Final Report of Internship Programme

The proposed system presents a practical, low-cost approach to thunderstorm prediction by combining live weather-data collection, feature engineering, and machine learning in a single integrated framework. Unlike conventional monitoring methods that rely only on static readings, this system uses multiple virtual sensor nodes and derives additional temporal features — pressure change and humidity change — to capture short-term atmospheric variation more effectively. The trained RandomForest model is connected to a real-time dashboard, making the prediction output easy to interpret and suitable for future expansion into smart weather-monitoring systems.

In the updated system, the novelty is further enhanced by the nearest safe-node recommendation layer on top of the prediction dashboard. This addition responds directly to evaluator feedback that the design would be more novel if it could also suggest the nearest safest place, and it moves the project from a descriptive interface toward actionable, decision-support guidance for users located at risky nodes.

5.4 Limitations

Although the project successfully demonstrates an end-to-end storm-prediction workflow, several limitations remain:

- The dataset is relatively small, which limits model generalisation.
- The class distribution is imbalanced, with few thunderstorm samples compared with non-thunderstorm samples.
- The high reported accuracy may reflect overfitting or a favourable train-test split rather than true robustness on unseen weather patterns.
- Real-time dashboard prediction still depends on the latest CSV records and does not yet include continuous streaming computation of change-based features from live sensors.
- The system uses virtual nodes and API data, so it does not yet represent a fully hardware-deployed embedded sensing network.

5.5 Future Scope

The project can be extended in several ways to increase its academic and practical value:

- Increase the dataset size by collecting data over a longer duration across more seasons and weather events.
- Handle class imbalance using resampling methods, class weights, or targeted data collection of thunderstorm events.
- Add further engineered features such as rolling averages, pressure-trend windows, humidity-trend windows, and location-aware statistics.
- Replace virtual nodes with ESP32-based physical weather stations connected to temperature, pressure, humidity, wind, and rainfall sensors.
- Improve the dashboard further with alert levels, confidence scores, historical plots, and automatic notifications.
- Retrain the model periodically as new weather data is appended to the CSV file.

6. Conclusion

Put together, this project ends up covering a fairly complete pipeline — pulling in distributed weather data, labelling it with a simple rule, engineering a couple of temporal features, training a RandomForest classifier, and visualising the result on a dashboard. What stands out from working through it is that even a fairly lightweight setup built

Final Report of Internship Programme

entirely on API-fed virtual nodes can do a decent job of localised weather-risk classification, and it captures the basic logic of what a full IoT-based monitoring system would look like.

The bigger change from the earlier version is the safe-node suggestion — TempestGuard doesn't just flag which nodes are at risk anymore, it also tells you where the nearest safe spot is. That came directly out of feedback on the first version, and it's arguably the part that makes the project feel less like a classroom exercise and more like something with practical value. Going forward, a larger dataset, better handling of the class imbalance, and eventually swapping the virtual nodes for real ESP32-based sensors would be the natural next steps toward a system that could genuinely support real-time thunderstorm warning and evacuation guidance.

7. AI Usage Declaration

AI tools were used at different stages of this project and its reporting. ChatGPT (OpenAI) and Google Antigravity were used during development for coding assistance, debugging, and technical research, including working with the OpenWeatherMap API, the RandomForest pipeline, and the Flask dashboard. Claude (Anthropic) was used during report preparation to reorganise the content into the required section structure (Introduction, Objective, Literature/Background, Methodology, Results and Discussion, Conclusion, References), to refine sentence-level phrasing and tense consistency, and to format the document according to the official cover-page template. The underlying project design, implementation, experiments, results, and figures are the author's own original work, and all AI-assisted code and text were reviewed and verified by the author before inclusion.

8. References

- [1] OpenWeatherMap, "Current Weather Data API," OpenWeatherMap. [Online]. Available: <https://openweathermap.org/current>
- [2] scikit-learn developers, "sklearn.ensemble.RandomForestClassifier," scikit-learn documentation. [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
- [3] Pallets Projects, "Flask Documentation," Flask. [Online]. Available: <https://flask.palletsprojects.com/>
- [4] Leaflet, "Leaflet — a JavaScript Library for Interactive Maps," Leaflet. [Online]. Available: <https://leafletjs.com/>